

Formative Assessment: Object-oriented Programming

Khiem Nguyen Lecturer in Multiscale Materials khiem.nguyen@glasgow.ac.uk

Table of contents

1 Foreword	1
2 Complex numbers	2
3 Fractions	3
4 Matrix	4
4.1 <code>xtensor</code> library	4
4.2 Quick revisit to Python	5
4.3 Implementation	6
5 One-dimensional Polynomial	6
5.1 Quick tour to <code>numpy.poly1d</code>	7
5.2 Implementation	8

1 Foreword

Solving these formative assignments will help you

- build muscle memory that will be useful for the paper exam in May
- enhance your skills and knowledge for assignment 02
- gain some little insight how widely used library such as NumPy is built. Of course, what we are doing here is not entirely done the same way in building such a massive library.

There will be four assignments and the first two are very similar to each other and the last two are inspired by `numpy.ndarray` and `numpy.Poly1D`.

1. Build a class representing fraction numbers
2. Build a class representing complex numbers
3. Build a class representing matrices in linear algebra
4. Build a class representing 1-dimensional polynomials

The first two assignments are taken directly from the mock exam last year and the past exam paper in 2025. The two assignments are so similar that you can practice with only one and move on to the more challenging assignment such as classes for matrices and for polynomials.

Note Try not to look at the solution from the mock exam and the past exam. Feel free to ask questions in the computer labs if needed. If you get stuck, go back to the lecture slides. In this way, you can build your muscle memory.

2 Complex numbers

In mathematics, a complex number is an element of a number system that extends the real numbers with specific element denoted i , called the imaginary unit and satisfying the equation $i^2 = -1$ (or equivalently $i = \sqrt{-1}$). Every complex number z can be expressed in the form $z = a + bi$, where $a \in \mathbb{R}$ and $b \in \mathbb{R}$ are real numbers. Two complex numbers $z_1 = a_1 + b_1i$ and $z_2 = a_2 + b_2i$ are added according to the rule

$$z_1 + z_2 = (a_1 + a_2) + (b_1 + b_2)i,$$

and multiplied by each other according to the rule

$$z_1 \cdot z_2 = (a_1 \cdot a_2 - b_1 \cdot b_2) + (a_1 \cdot b_2 + a_2 \cdot b_1)i.$$

Your task is to write C++ code to define a class `ComplexNumber`, its data members, member functions, and non-member functions (if there are) to reflect the above description. You should see (a) – (h) questions.

1. Write code to define the class `ComplexNumber` with two private data members, named `re` and `im`.
2. Write a constructor receiving two input arguments `re` and `im` with default value 0 to initialize the data members described in (a). Use member initializer list (not the assignment statements) to initialize these data members.
3. Write a function overloading the operator `<<` of the object `std::ostream` so that the statement `std::cout << comp_num;` works for the `ComplexNumber` object `comp_num`. You must decide the print-out format that makes sense to the user. For example, consider the following numbers: $1 + 0i = 1$ and $0 + 2i = 2i$.
4. Write functions to overload the operator `+` (plus sign) and the operator `*` (multiplication sign) to reflect the addition and multiplication of two complex numbers, respectively. Essentially, the following statements should work after overloading the operators `<<`, `+` and `*` for the complex numbers:

```
std::cout << a + b;  
std::cout << a * b;
```

5. Write a member function with name `pow` to compute the power z^n of the complex number z with n being a non-negative integer. Recall that

$$z^0 = 1, \quad z^1 = z, \quad z^2 = z \cdot z, \quad z^3 = z \cdot z \cdot z, \quad z^n = \underbrace{z \cdot \dots \cdot z}_{n \text{ terms}}.$$

6. Write a function to overload the assignment operator `=` for a `ComplexNumber` object so that the following statement works:

```
c = a + b; // a, b, c are all ComplexNumber objects
```

7. Write a class `RealNumber` derived from the class `ComplexNumber` you have defined above. The class `RealNumber` must have a constructor receiving one `double` number and make sure that the imaginary part derived from the parent class is equal to zero.
8. Write a `main` function to perform the following tasks:
 - Create an array of five `ComplexNumber` elements using dynamic memory allocation with a pointer to `ComplexNumber`. Remember to clean up the memory when you do not need the array anymore.
 - Initialize these five complex numbers using the `for`-loop to obtain five numbers (note the pattern of the real parts and imaginary parts of these numbers):

$$z_1 = 1 + 2i, \quad z_2 = 2 + 3i, \quad z_3 = 3 + 4i, \quad z_4 = 4 + 5i, \quad z_5 = 5 + 6i.$$

4. Compute the sum $s = \sum_{j=1}^5 z_j$ of these five numbers using the `for`-loop and print the resulted sum s to the console.

5. With $z = 1 + 2i$, compute the expression

$$\Lambda = 1 + z + z^2 + z^3 + z^4 + z^5$$

using the `for`-loop.

Note that you can write any helper functions that are not defined by answering the above questions to complete the *main function and other functions*.

Hint `static_cast<double>(12)` will convert the `int` 12 into the `double` 12.0.

3 Fractions

Your task is to write C++ code to define a class `Fraction`, its data members, member functions, and non-member functions (if applicable) to fulfill the following description.

1. An object of type `Fraction` has two private data members of integer type, named `nom` and `denom`.
2. Write a constructor receiving two input arguments `nom` and `denom` with appropriate default values to initialize the data members. Use a member initializer list (not assignment statements). Ensure that the denominator cannot be zero: provide a warning and set the denominator to 1 if 0 is provided.
3. Write a member function to perform the addition of two fractions. You need to decide what the inputs and outputs of this function should be.
4. Write a member function to perform the multiplication of two fractions.
5. Overload the operator `<<` for `std::ostream` so that `std::cout << f;` works for a `Fraction` object. The function must handle the following formatting:

- If the numerator is 0, print 0.
- If the denominator is 1, print only the numerator (e.g., 42/1 prints as 42).
- If $f > 0$, print the fraction in the form a/b (e.g., 2/3 for both 2/3 and $-2/-3$).
- If $f < 0$, print the negative sign in front of the positive fraction (e.g., 2/-3 prints as -2/3).

6. Overload the operator `+` (plus) and operator `*` (multiplication) so that the following statements work for two `Fraction` objects `a` and `b`:

```
std::cout << a + b; // Equivalent to std::cout << a.add(b)
std::cout << a * b; // Equivalent to std::cout << a.multiply_by(b)
```

7. Overload the assignment operator `=` for a `Fraction` object so that the following works:

```
c = a + b; // a, b, and c are all Fraction objects
```

8. Write a member function `reduce` that reduces a fraction to its irreducible form using `std::__gcd(int, int)` from `#include <bits/stdc++.h>`. For example, the GCD of 100 and 40 is 20, so the fraction should be reduced accordingly.

9. Finally, write a `main` function to perform the following:

- Create a `std::vector` containing five `Fraction` objects with the values:

$$f_1 = \frac{1}{2}, \quad f_2 = \frac{1}{3}, \quad f_3 = \frac{1}{4}, \quad f_4 = \frac{1}{5}, \quad f_5 = \frac{1}{6}$$

using a `for`-loop and `push_back()`.

- Compute the product using a `for`-loop:

$$p = \prod_{i=1}^5 f_i = f_1 \cdot f_2 \cdot f_3 \cdot f_4 \cdot f_5 = \frac{1}{2 \cdot 3 \cdot 4 \cdot 5 \cdot 6}$$

- Print the resulting product to the console.

4 Matrix

We know that the library NumPy in Python allows us to create, perform computations on high-dimensional arrays. However, to build such a complex library needs a lot of man power, skills, experience, and definitely beyond our scope in this course. In fact, the lecturer of this course cannot build this library all alone. For this reason, we reduce our ambition and focus on mimicing two-dimensional arrays in NumPy, that is equivalent to Matrices. In this project, you try to decide your own design. You can consult Gemini/ChatGPT what is the best to do. Below are some of my suggestions for this project.

4.1 xtensor library

There has been some serious effort of building C++ library that mimic NumPy library as much as possible: [xtensor](#). [xtensor](#) is a C++ library meant for numerical analysis with multi-dimensional array expressions. Below are some examples quoted directly from its GitHub's README.md

- **Basic usage**

```
#include <iostream>
#include "xtensor/xarray.hpp"
#include "xtensor/xio.hpp"
#include "xtensor/xview.hpp"

xt::xarray<double> arr1
{{1.0, 2.0, 3.0},
 {2.0, 5.0, 7.0},
 {2.0, 5.0, 7.0}};

xt::xarray<double> arr2
{5.0, 6.0, 7.0};

xt::xarray<double> res = xt::view(arr1, 1) + arr2;

std::cout << res;
```

- **Index Access**

```
#include <iostream>
#include "xtensor/xarray.hpp"
#include "xtensor/xio.hpp"

xt::xarray<double> arr1
{{1.0, 2.0, 3.0},
 {2.0, 5.0, 7.0},
 {2.0, 5.0, 7.0}};

std::cout << arr1(0, 0) << std::endl;

xt::xarray<int> arr2
{1, 2, 3, 4, 5, 6, 7, 8, 9};

std::cout << arr2(0);
```

The reference to [xtensor](#) library is just for your awareness. Building a library like this is definitely beyond the scope of this course; it demands a lot of man power, and thus many contributions (see its GitHub page).

4.2 Quick revisit to Python

Import the library

```
import numpy as np
```

Create matrices

We can create matrices in NumPy using constructor `numpy.array()` or using array creation routines such as `np.ones()`, `np.zeros()` and so on – See the list [here](#)

```
a = np.array([[1, 2, 3], [4, 5, 6]], dtype=np.float64)
b = np.ones(shape=(2, 3))
c = np.linspace(1.5, 7.5, 6).reshape((2, 3))
```

Arithmetic operations

Then, we can perform arithmetic operations on two-dimensional arrays:

```
print(f"a + b =\n {a + b}")
print(f"a - b =\n {a - b}")
print(f"a * c =\n {a * c}")
print(f"a / c =\n {a / c}")
```

```
a + b =
[[2. 3. 4.]
 [5. 6. 7.]]
a - b =
[[0. 1. 2.]
 [3. 4. 5.]]
a * c =
[[ 1.5  5.4 11.7]
 [20.4 31.5 45.  ]]
a / c =
[[0.66666667 0.74074074 0.76923077]
 [0.78431373 0.79365079 0.8        ]]
```

We can also perform arithmetic operations between a matrix and an scalar

```
print(f"1 + a =\n {1 + a}")
print(f"2.5 * a =\n {2.5 * a}")
```

```
1 + a =
[[2. 3. 4.]
 [5. 6. 7.]]
2.5 * a =
[[ 2.5  5.  7.5]
 [10. 12.5 15.  ]]
```

Transpose

We can transpose a matrix

```
print(f"a transpose = \n{a.transpose((1, 0))}")
```

```
a transpose =  
[[1. 4.]  
 [2. 5.]  
 [3. 6.]]
```

4.3 Implementation

- Thinking about what you have learned about NumPy array, you can design a class `Matrix` that mimic the concept of 2-dimensional NumPy array. In addition, you should overload arithmetic operators.
- Since we create matrices by using `numpy.array()` and call functions such as `numpy.ones()` and `numpy.zeros()` by accessing the module `numpy`, this concept can be mapped to C++ implementation via namespace. To realize this, we can put the entire class `Matrix` and helper functions, such as `zeros()` and `ones()` returning a matrix, in the namespace `numpy`.
- To be able to use the statement `std::cout << matrix;`, you need to overload the `operator<<`.
- In order to create a `Matrix` object using initialization list as below:

```
#include "Matrix.hpp"  
namespace np = numpy;  
  
int main() {  
    // Creating a 2D matrix  
    auto mat2d = np::array({  
        {1.1, 2.2, 3.3},  
        {4.4, 5.5, 6.6}  
    });  
  
    // Creating a 1D matrix (row vector)  
    auto mat1d = np::array({10.0, 20.0, 30.0});  
  
    std::cout << "2D Matrix:" << std::endl;  
    std::cout << mat2d;  
  
    return 0;  
}
```

We must use `{.cpp}` `std::std::initializer_list`

- If you want to generalize your class `Matrix` so that it can accept different basic data types such as `int`, `float`, and `double`, you need to create a class template.
- Test your code with the above `main` function.

5 One-dimensional Polynomial

NumPy library provides us a powerful and useful class representing one-dimensional polynomial: `numpy.poly1d`. In this project, we try to build a prototype of this class and equip it with some basic operations. To build intuition of what to do, let us make a quick tour to `numpy.poly1d` by following simple examples below.

5.1 Quick tour to `numpy.poly1d`

Import the library Class `numpy.poly1d` is provided in the module `numpy`. So we import `numpy` as before.

```
import numpy as np
```

Create a 1-dimensional polynomial

```
p1 = np.poly1d([1, 2, 3, 4, 5])
p2 = np.poly1d([5, 4, 3])
print(f"p1 =\n{p1}")
print(f"p2 =\n{p2}")
```

```
p1 =
      4      3      2
1 x + 2 x + 3 x + 4 x + 5
p2 =
      2
5 x + 4 x + 3
```

Arithmetic operations

We can perform arithmetic operations on 1-dimensional polynomials:

```
q1 = p1 + p2
q2 = p1 - p2
r = p1 * p2
print(f"q1 =\n{q1}")
print(f"q2 =\n{q2}")
print(f"r =\n{r}")
```

```
q1 =
      4      3      2
1 x + 2 x + 8 x + 8 x + 8
q2 =
      4      3      2
1 x + 2 x - 2 x + 2
r =
      6      5      4      3      2
5 x + 14 x + 26 x + 38 x + 50 x + 32 x + 15
```

Again, we can add a scalar to a polynomial

```
s = 1 + p1
print(f"s =\n{s}")
```

```
s =
      4      3      2
1 x + 2 x + 3 x + 4 x + 6
```

Evaluation at one value x

Of course, we want to evaluate a polynomial at one coordinate x

```
print(f"p1(2) = {p1(2)}")
```

```
p1(2) = 57
```

5.2 Implementation

- Thinking about what you have learned about NumPy array, you can design a class `Poly1D` that mimic the class `numpy.poly1d`. You should overload arithmetic operators.
- Again, it is clear that the class `Poly1D` should be put in the namespace `numpy`.
- To be able to use the statement `std::cout << polynomial;`, you need to overload the `operator<<`.
- You should overload the `operator()` to realize the function call `polynomial(2)`.